

PRD: Observability Implementation Lab

Building a Modern MELT Operating Layer for Containerized Services

1. Summary

Each group must add a modern observability layer to its existing Docker Compose-based service architecture.

The goal is to move from:

“Our services are running.”

to:

“Our services can be observed, tested, alerted on, traced, and debugged by another engineer.”

Each group will implement the **MELT model**:

Signal	Purpose
Metrics	Show service health, traffic, errors, and latency
Events	Show important operational moments and changes
Logs	Explain what happened inside each service
Traces	Show the request journey across services

This is not a dashboard decoration assignment.

This is an operational visibility assignment.

2. Product Goals

Goal 1: Create one operating view

Use **Grafana** as the central observability cockpit where possible.

Grafana should help reviewers see:

- Service health
- Request rate
- Error rate
- p95 latency

- • Alert state
- • Prometheus metrics
- • Jaeger trace links or trace IDs
- • Optional logs through Loki

The goal is not to force one tool to do everything. The goal is to create one operating view backed by specialized open-source tools.

Goal 2: Instrument services properly

Each service must expose enough telemetry to explain its behavior.

At minimum, each service must support:

- • /health
- • /metrics
- • Structured JSON logs
- • Trace spans for incoming requests
- • Trace propagation when calling another service

This matters because production failures often happen across service boundaries. If service-a calls service-b, and service-b calls service-c, engineers should be able to follow that path.

Goal 3: Add load testing and benchmarks

Each group must prove how the system behaves under:

- • Normal traffic
- • Stress traffic
- • Controlled failure

The load test must be repeatable. A reviewer should be able to run one command and see metrics, logs, traces, or alerts change.

Goal 4: Add alerting

Each group must define alert conditions for real operational problems.

Minimum required alerts:

- • Service down
- • High error rate
- • High latency

Alerts must include clear reproduction and verification steps.

3. Required Architecture

Update your Docker Compose setup to include application services and observability services.

Minimum Architecture

Client / Load Test Tool

|

v

Nginx / Gateway

|

v

Service A ---> Service B ---> Service C

| | |

| | |

+---- Metrics --+----- Metrics --+

| | |

+---- Logs -----+----- Logs -----+

| | |

+---- Traces --+----- Traces --+

|

v

Prometheus <--- metrics scraping

Jaeger <--- distributed traces

Grafana <--- central operating view

Required Containers

Your docker-compose.yml must include:

service-a

service-b

service-c

nginx or gateway

prometheus

jaeger

grafana

Strongly Recommended, But Optional

alertmanager

loki

promtail

cadvisor

node-exporter

otel-collector

Only add optional tools if your README explains:

What problem the tool solves

What data it collects

Where the data is viewed

How it helps debugging

4. Required Open-Source Stack

4.1 Prometheus — Metrics

Prometheus must collect metrics from each service.

Each service must expose a Prometheus-compatible /metrics endpoint.

Prometheus must scrape services using Docker Compose service names, not hardcoded localhost assumptions.

Example:

scrape_configs:

- **job_name:** "service-a"

- static_configs:**

- **targets:** ["service-a:3000"]

- **job_name:** "service-b"

- static_configs:**

- **targets:** ["service-b:3000"]

- **job_name:** "service-c"

- static_configs:**

- **targets:** ["service-c:3000"]

Prometheus must use a persistent named volume.

Example:

volumes:

- prometheus-data:**

Why this matters:

Prometheus helps answer:

Is the service up?

How many requests are coming in?

How many requests are failing?

Is latency getting worse?

Did the service return to normal?

4.2 Grafana — Central Operating View

Grafana must connect to Prometheus as a data source.

Recommended dashboard panels:

- • Service up/down status

- • Request rate
- • Error rate
- • p95 latency
- • Alert state
- • Optional container metrics
- • Optional logs via Loki

Why this matters:

During debugging, engineers should not rely only on random terminal commands and guesswork. Grafana should make system state visible quickly.

4.3 Jaeger — Distributed Tracing

Jaeger must be added for tracing.

Each service should emit trace spans. Where one service calls another, trace context should be propagated.

Minimum expected trace path:

gateway

→ **service-a**

→ **service-b**

→ **service-c**

Each span should show:

service name

endpoint

duration

status

error state if failed

Required trace demo:

1. **Send a successful request through the gateway**
2. **Open Jaeger**
3. **Search by service name**

4. Show the full request journey

5. Trigger a slow or failing endpoint

6. Show where latency or failure appears in the trace

Why this matters:

Prometheus can show that latency increased. Jaeger helps show where the latency happened.

4.4 Logs — Structured Service Evidence

Each service must emit structured JSON logs.

Minimum request log:

```
{  
  "timestamp": "2026-07-08T19:35:00Z",  
  "service": "service-a",  
  "level": "info",  
  "method": "GET",  
  "path": "/api/service-a",  
  "status": 200,  
  "duration_ms": 34,  
  "request_id": "abc-123",  
  "trace_id": "trace-123"  
}
```

Minimum error log:

```
{  
  "timestamp": "2026-07-08T19:36:00Z",  
  "service": "service-b",  
  "level": "error",  
  "error": "dependency timeout",  
  "request_id": "abc-123",  
}
```

```
"trace_id": "trace-123"  
}
```

Required:

Every request has a request_id

Logs include service name

Logs include log level

Logs include request duration where applicable

Error logs are easy to identify

Trace ID is included where possible

Minimum acceptable log access:

docker compose logs service-a

Recommended:

Loki + Promtail for viewing logs inside Grafana

Why this matters:

Metrics show the symptom.

Traces show the path.

Logs explain the local detail.

4.5 Events — Operational Moments

Each group must document at least three meaningful operational events.

Examples:

Deployment started

Deployment completed

Load test started

Load test completed

Failure triggered

Alert fired

Configuration changed

Events may be represented through:

- • Structured logs
- • Grafana annotations
- • Alert events
- • README notes
- • Benchmark report timeline

Why this matters:

Events help engineers connect system behavior with important changes.

4.6 Alerting — Detection Layer

Each group must define at least three alert conditions.

Required alerts:

Alert 1: Service Down

`up{job="service-a"} == 0`

Alert 2: High Error Rate

`rate(http_errors_total[2m]) > 0.1`

Alert 3: High Latency

`histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m])) > 0.5`

Each alert must document:

Alert name

PromQL condition

What it means

Possible causes

How to reproduce it

First checks

How to confirm normal state

Minimum acceptable:

Prometheus alert rules with documented manual verification

Recommended:

Grafana alerts, Alertmanager, Slack, Discord, email, or webhook notification

Why this matters:

Humans should not discover service problems by accident. The system should expose meaningful failure conditions clearly.

5. Service Instrumentation Requirements

Each service must implement these endpoints.

/health

Returns service health.

Example:

```
{  
  "service": "service-a",  
  "status": "ok",  
  "dependencies": {  
    "service-b": "ok"  
  }  
}
```

If a dependency is unavailable:

```
{  
  "service": "service-a",  
  "status": "degraded",  
  "dependencies": {  
    "service-b": "unreachable"  
  }  
}
```

Why this matters:

Health checks help engineers and alerting systems distinguish between healthy, degraded, and failed states.

/metrics

Exposes Prometheus metrics.

Minimum required metrics:

http_requests_total

http_request_duration_seconds

http_errors_total

service_up

Recommended labels:

service

method

route

status_code

Bad metric:

requests_total

Better metric:

http_requests_total{service="service-a", route="/api/service-a", method="GET", status_code="200"}

Why this matters:

Labels help engineers isolate problems by service, route, method, or status code.

Controlled Failure Endpoints

Each group must create at least one controlled failure or degradation endpoint.

Examples:

/fail

/slow

/error

/dependency-fail

These endpoints exist for testing observability and alerting.

They must be clearly marked as lab-only or test-only.

Why this matters:

You cannot prove observability without safely creating a known failure.

6. Load Testing and Benchmarking

Each group must include a repeatable load test.

Allowed tools:

- k6
- hey
- wrk
- ab
- custom script

Recommended:

- k6

Required benchmark scenarios:

Scenario	Purpose
Normal traffic	Establish baseline behavior
Stress traffic	Observe latency and error behavior under pressure
Failure traffic	Prove alerts, traces, and logs work during failure

Required file:

docs/benchmark-report.md

Benchmark report must include:

Scenario	Requests	Concurrency	Avg Latency	p95 Latency	Error Rate
Normal traffic	500	10	40ms	120ms	0%
Stress traffic	2000	50	180ms	650ms	2%

Failure traffic	300	10	N/A	N/A	100%
-----------------	-----	----	-----	-----	------

Why this matters:

Observability should show system behavior changing under load. If your metrics look the same before, during, and after load testing, your instrumentation is weak.

7. Failure Simulation

Each group must simulate one controlled failure.

Choose one:

Failure A: Service Down

docker compose stop service-b

Expected evidence:

Prometheus target goes down

Alert condition becomes true

Dependent service shows degraded health

Logs show failed dependency

Trace shows request failure path

Failure B: High Latency

/service-b/slow

Expected evidence:

p95 latency increases

Latency alert becomes true

Jaeger shows slow span

Logs show slow request duration

Benchmark report captures degraded performance

Failure C: High Error Rate

/service-c/fail

Expected evidence:

Error counter increases

Error rate alert becomes true

Logs show structured error entries

Trace shows failed service span

8. Required Repository Structure

Each group must submit an updated GitHub repository with this structure or a clearly documented equivalent:

.

|—— **docker-compose.yml**

|—— **prometheus.yml**

|—— **alert-rules.yml**

|—— **README.md**

|—— **grafana/**

| |—— **dashboards/**

| |—— **provisioning/**

|—— **jaeger/**

| |—— **README.md**

|—— **scripts/**

| |—— **load-test.js or load-test.sh**

|—— **docs/**

```
| |—— architecture.md
| |—— benchmark-report.md
|—— services/
    |—— service-a/
    |—— service-b/
    |—— service-c/
```

9. Required Documentation

README.md

Must include:

How to start the stack

How to stop the stack

How to access each service

How to access Prometheus

How to access Grafana

How to access Jaeger

How to run load tests

How to trigger failure

How to view metrics

How to view traces

How to view logs

How to confirm alerts

docs/architecture.md

Must include:

Service architecture

Request flow

Telemetry flow

Metrics collection flow

Tracing flow

Logging flow

Alerting flow

Known limitations

Include an ASCII diagram or image.

docs/benchmark-report.md

Must include:

Test tool used

Test command

Normal traffic results

Stress traffic results

Failure traffic results

Metrics observed

Alerts triggered

Traces observed

Lessons learned

10. Acceptance Criteria

Requirement	Acceptance Criteria
Stack startup	docker compose up starts application and observability services
Prometheus	Prometheus can scrape all services
Persistent metrics	Prometheus uses a named volume

Metrics	Services expose request count, error count, and latency
Grafana	Grafana connects to Prometheus
Jaeger	Jaeger receives traces from at least two services
Trace propagation	One request can be followed across multiple services
Logs	Services emit structured JSON logs
Request correlation	Logs include request ID and preferably trace ID
Health checks	Each service exposes /health
Alerts	At least 3 alert rules exist and are documented
Load testing	A repeatable load test script exists
Benchmarks	Benchmark report includes normal, stress, and failure scenarios
Failure simulation	One failure is triggered and explained using MELT signals

11. Review Scorecard

Area	Points	What Reviewers Should Check
Architecture clarity	10	Request flow and telemetry flow are documented
Docker Compose setup	10	App and observability services run together
Prometheus metrics	15	Useful metrics are exposed, scraped, and queryable
Grafana operating view	10	Dashboard helps explain health, errors, and latency
Jaeger tracing	15	Request path is visible across services
Structured logs	10	Logs are readable, correlated, and useful
Alerting	10	Alerts detect real service down, error, and latency issues
Load testing	10	Benchmarks are repeatable and documented
Failure simulation	5	Failure is triggered and explained using MELT signals

Documentation quality	5	Another engineer can run and review without hand-holding
Total	100	

12. Saturday Demo Flow

Each group must prepare a 7-minute demo.

Recommended flow:

- 1. Show architecture diagram**
- 2. Start the stack**
- 3. Open Grafana**
- 4. Open Prometheus targets**
- 5. Open Jaeger**
- 6. Send a successful request**
- 7. Show metrics changing**
- 8. Show trace path across services**
- 9. Run load test**
- 10. Trigger controlled failure**
- 11. Show alert condition**
- 12. Use metrics, logs, and traces to explain failure**
- 13. Confirm system returns to normal**
- 14. Share one production improvement**

The demo should prove operational understanding, not just tool installation.

13. Definition of Done

Your group is done when:

The stack starts cleanly

Services respond through the gateway

Prometheus scrapes all services

Grafana shows service health and performance

Jaeger shows at least one multi-service trace

Logs are structured and useful

Load testing produces measurable changes

At least one alert condition can be triggered

One controlled failure is simulated and explained

The README allows another group to reproduce your work

14. Final Standard

By the end of this assignment, each group should understand this operating principle:

Metrics tell us something is wrong.

Logs explain what happened.

Traces show where it happened.

Events show what changed.

Alerts call attention to the issue.

The goal is not just to run containers.

The goal is to operate services with visibility and discipline.